



BSc Computer Science

CM2005 Object Oriented Programming

End-of-term coursework assignment

Introduction

In this course, we have developed a basic DJ application called Otodecks. For the end-of-term assessment, you are tasked with developing the application further by adding functionalities to support a DJ in preparing and doing live mixing of the tracks. The DJ application should be able to load music files into the music library or directly to the decks to play the music on the relevant deck and other specific requirements listed below. Update the user interface and music decks in your own interesting way that is easy to use. The DJ application should persist information between application loads, so it will need to store its state and configurations (*i.e., music library, track-based hot cues and equaliser*) in one or more data files with a suitable data structure.

Requirements

We will assess your work based on the following requirements and criteria:

R1: The DJ application should contain the following core functionality, as shown in class

- R1A: Load audio files into audio players.
- R1B: Play two or more tracks simultaneously.
- R1C: Mix track by varying each of their volumes.
- R1D: Speed up and slow down the tracks.

R2: The music library component should allow the user to manage their music and persist even after the application has closed and reopened.

- R2A: music library component should allow the user to load multiple files into the deck and display the track details (*i.e., file name, durations, ...*) in the playlist.
- R2B: load music from the library to different decks.
- R2C: music library state (*i.e., playlist with track details from R2A*) should persist after re-opening the application.

R3: Create hot cue buttons (*up to 8*) that you can trigger instantly and jump to a particular time point in the track (*i.e., at the vocal introductions, drum breaks, or drops for easier live remixing*). These hot cue buttons are very useful for the DJ to do live mixing of tracks and creativity.

- R3A: Assign hot cues using the buttons while playing the track or when paused.
- R3B: Allow the user to modify the cues individually.
- R3C: Allow the user to clear all hot cues using a dedicated button or similar.
- R3D: Persist and load the hot cues for each track when re-opening the application.

R4: Develop a three-band equaliser (EQ) mixer (*i.e.*, *high*, *mid* and *low*) panel for each deck using the [DSP module](#) within the JUCE library.

- R4A: Create three knobs or sliders to allow the user to adjust the equaliser value of a track to high, medium and low.
- R4B: Persist the track's original and the last adjusted EQ setting information when closing and re-opening the application.

R5: Investigate and discuss what and how Beats Per Minute (BPM) information for a track is used within the DJ application context.

- R5A: Discuss what and how BPM is used within a DJ application. Provide at least two examples of scenarios in which they will be used and how.
- R5B: Highlight alternative techniques (or features available in popular DJ applications) that can be used for the purpose and example discussed in R5A.
- R5C: Conceptually discuss how you would calculate the BPM value for a given track.
- R5D: Implement and visualise the BPM value for a track within the application. If you do not implement and visualise it, at least technically discuss how you would implement it.

Code style and technique

Your code should be written according to the following style and technique guidelines:

C1: Code is organised into header (.h or .hpp) files and implementation files (.cpp). Header files contain class interface definitions, cpp files contain implementations of class function members.

C2: Class interfaces in header files have comments for each public function describing purpose, inputs and outputs.

C3: Code is laid out clearly with consistent indenting.

C4: Code is organised into functions with clear inputs and outputs and a clear, limited purpose.

C5: Code is stateless wherever possible – functions make use of data passing in preference to global or class scope data.

C6: Functions, classes and variables have meaningful names, with a consistent naming style.

C7: Functions do not change the state of class or global scope variables unless that is the explicit purpose of a function (*e.g.*, a *setter*).

Report

A report in a PDF format is required to document how **ALL** the requirements are implemented, with:

- i. a brief discussion on the code logic (*include fragments of code if needed*).
- ii. any assumptions, decisions, or unique features or behaviours of the application included
- iii. supporting screenshots of the final outcome (*with suitable captions/annotations where relevant*).

Video demonstration

A video demonstrating your program in an **MP4** format, maximum 5 minutes. The video should contain the following:

- You briefly reviewing your code in your IDE, then launching the program (*ensure the IDE and terminal are visible in the background throughout the video*).
- You narrating (*audibly, not computer-generated voice or text*) what the code is doing as you click on buttons and other user interface elements.
- Demonstration of functionality pertaining to **ALL** the requirements. More specifically, show playback controls features of the decks (R1), showing tracks loading into the library (R2), creating and managing cues (R3), equaliser filter panel (R4) and discussing or showcasing the BPM feature implemented (R5) to smoothly transition two tracks.

Code PDF

Create a PDF document by pasting the code of the application as text and **annotating ALL sections you have personally written without assistance** clearly. If you are using the template codes provided, please omit these sections where relevant in this code PDF. Code implementation marks will **ONLY** be awarded on the code you have written.

Marking criteria

We will mark your work according to the set of criteria shown below, which considers the requirements, your programming technique and style and the documentation you have provided:

Tasks	Points
Submit correct files: code as text in PDF, report as PDF, a ZIP file for the code & a video*.	4
Code style: indentation, descriptive comments	10
R1: Basic Program	10
<i>R1A: Load audio files into audio players</i>	2
<i>R1B: Play two or more tracks simultaneously</i>	2
<i>R1C: Mix track by varying each of their volumes</i>	3
<i>R1D: Speed up and slow down the tracks.</i>	3
R2: Music Library & Persisting	10
<i>R2A: music library component should allow the user to load multiple files into the deck and display the track details (i.e., file name, durations, ...) in the playlist.</i>	4
<i>R2B: load music from the library to different decks.</i>	3
<i>R2C: music library state should persist even after re-opening the application.</i>	3
R3: Hot Cues Buttons & Persisting	10
<i>R3A: Assign hot cues using the buttons while playing the track or when paused.</i>	3
<i>R3B: Allow the user to modify the cues individually.</i>	2
<i>R3C: Allow the user to clear all hot cues using a dedicated button or similar.</i>	2
<i>R3D: Persist and load the hot cues for each track when re-opening the application.</i>	3
R4: Develop a Tri-band Equaliser Panel with High, Mid and Low filter options.	10
<i>R4A: Create three knobs or sliders to allow the user to adjust the equaliser value of a track to high, medium and low.</i>	7
<i>R4B: Persist the track's original and the last adjusted EQ setting information when closing and re-opening the application.</i>	3
R5: Investigate the purpose of the Beats Per Minute (BPM)	15
<i>R5A: Discuss what and how BPM is used within a DJ application. Provide at least two examples of scenarios in which they will be used and how.</i>	4
<i>R5B: Highlight alternative techniques (or features available in popular DJ applications) that can be used for the purpose and example discussed in R5A.</i>	2
<i>R5C: Conceptually discuss how BPM values are calculated for a given track.</i>	4
<i>R5D: Implement and visualise the BPM value for a track within the application. If not, at least technically discuss how you would implement it.</i>	5
Report	25
The report includes a brief discussion on the code logic (<i>including fragments of code if needed</i>) for ALL requirement.	10
The report includes any assumptions, decisions, or unique features or behaviours of the application included	10
The report includes supporting screenshots of the final functionalities developed.	5
Video	6
TOTAL	100

What you should submit

You need to submit the following items:

- **Source code** in standard **ZIP** format.*
- **Code in PDF** format.* It should contain all the code formatted with clear comments showing which parts of the code you personally wrote without assistance.
- **Report in PDF** format.*
- **Video demonstrating** your program in an **MP4** format. Maximum 5 minutes.

* **Please note: If you do NOT submit CODE PDF AND REPORT PDF files along with the program files in a ZIP file, NO MARKS will be awarded for the respective tasks for the CODE implementation marking criteria. General university regulations, assessment policy and generative AI policy apply.**

[END OF COURSEWORK ASSIGNMENT]